

Yenepoya (Deemed to be University)
Final Project Report
on
Honeypot Network for Agriculture Threat
Intelligence

Student Name: Farhan N Methar

Project IDPRJN26-205

Industry Mentor: SHASHANK

Guided by: SUZANA SHAROL

Table of Contents

- 1. Background**
- 2. Aim**
- 3. Technologies**
- 4. Hardware Architecture**
- 5. Software Architecture**
- 6. System Requirements**
 - 6.1. Functional Requirements**
 - 6.2. User Requirements**
 - 6.3. Environmental Requirements**
- 7. Design and Architecture**
- 8. Implementation**
- 9. Testing**
 - 9.1. Graphical User Interface (GUI) Layout**
 - 9.2. Customer Testing**
- 10. Evaluation**
- 11. Snapshots of the Project**
- 12. Conclusions**
- 13. Further Development or Research**
- 14. References**
- 15. Appendix**

Executive Summary

• Background

Agriculture is increasingly dependent on smart IoT devices such as irrigation controllers, soil sensors, and environmental monitors. These systems are vulnerable to unauthorized access and cyberattacks. Traditional cybersecurity tools focus on urban or enterprise environments, leaving agriculture-specific systems largely unprotected.

• Aim

The aim of this project is to build a Python-based Honeypot Network that simulates an AgriSense IoT Gateway device, monitors unauthorized connection attempts, logs threat intelligence data, and visualizes attack patterns using IBM Cognos Analytics.

• Technologies

- Python 3 — Core programming language
- Socket Library — TCP server and connection handling
- Threading — Simultaneous connection handling
- CSV — Logging and data storage
- IBM Cognos Analytics — Dashboard and visualization
- VS Code — Development environment
- GitHub — Version control and project repository

• Hardware Architecture

The system runs entirely on a standard laptop or desktop computer. No special hardware is required. The honeypot simulates an AgriSense IoT Gateway device in software, making it accessible and easy to deploy.

• Software Architecture

The system consists of three Python scripts and two output files. `honeypot.py` is the main server that accepts connections, logs data, detects suspicious activity, and blocks malicious IPs. `client.py` and `client2.py` are test scripts that simulate connection attempts. All logs are saved in `log.csv` and visualized in IBM Cognos Analytics.

1. Background

Agriculture is one of the most critical sectors in the world, and it is rapidly adopting Internet of Things (IoT) technology. Devices such as smart irrigation systems, soil moisture sensors, weather monitors, and crop management systems are now connected to the internet. While these technologies improve farming efficiency, they also introduce new cybersecurity risks.

A Honeypot is a cybersecurity trap — a decoy system designed to attract attackers. When an attacker connects to the honeypot, their IP address, port number, and timestamp are logged. This data is used to understand attack patterns and build threat intelligence.

This project applies the honeypot concept to an agriculture context by simulating an AgriSense IoT Gateway device. The system monitors unauthorized access attempts and helps build Agriculture-specific Threat Intelligence — a relatively unexplored area in cybersecurity.

2. Aim

The primary aims of this project are:

- To build a functional honeypot using Python's Socket library that listens on port 8888
- To simulate an AgriSense IoT Gateway device to attract and trap unauthorized connections
- To log all connection attempts with details including IP address, port, timestamp, and status
- To detect suspicious activity using threshold-based logic (5+ attempts = Suspicious)
- To permanently block suspicious IPs and persist the block list across server restarts
- To handle multiple simultaneous connections using threading with thread-safe locks
- To visualize threat intelligence data using IBM Cognos Analytics dashboard
- To document the project with a complete HLD, LLD, and Final Report

3. Technologies

Technology	Purpose
Python 3	Core programming language for the entire system
socket library	Creates TCP server, accepts connections, sends responses
threading library	Handles multiple simultaneous connections
threading.Lock()	Prevents race conditions in shared data access
csv library	Writes connection logs to log.csv
datetime library	Captures timestamp of each connection attempt
os library	Checks file existence for CSV and blocked IPs file
IBM Cognos Analytics	Visualizes log data with charts and dashboards
VS Code	Code editor used for development
GitHub	Version control and project hosting

4. Hardware Architecture

This project does not require any specialized hardware. It runs entirely on a standard laptop or desktop computer with the following minimum specifications:

Requirement	Specification
Operating System	Windows 10 / 11, macOS, or Linux (Ubuntu)
Processor	Any modern CPU (Intel Core i3 or equivalent)
RAM	Minimum 4 GB
Storage	Minimum 100 MB free space for logs and scripts
Network	Local network or localhost for testing
Python Version	Python 3.8 or above

The honeypot simulates an AgriSense IoT Gateway entirely in software. No physical IoT device is required. This makes the project highly portable and easy to test on any standard computer.

5. Software Architecture

The system is built using three Python scripts and produces two output files:

File	Description
honeypot.py	Main honeypot server — listens on port 8888, logs connections, detects and blocks suspicious Ips
client.py	Test client — connects once to the honeypot to simulate a single connection attempt
client2.py	Test client — connects 7 times to simulate repeated attack attempts and trigger detection
log.csv	Output file — stores all connection logs (Time, IP, Port, Attempts, Status)
blocked_ips.txt	Output file — stores permanently blocked IP addresses, persists across restarts

The software architecture follows a server-client model. The honeypot server runs continuously, accepts connections, and processes each one in a separate thread. All shared data is protected by a threading lock to prevent race conditions.

6. System Requirements

6.1 Functional Requirements

- The system must listen for incoming TCP connections on port 8888
- The system must log the IP address, port, timestamp, and attempt count of every connection
- The system must send a fake AgriSense IoT Gateway response to every connecting client
- The system must detect when an IP has made 5 or more connection attempts
- The system must flag repeated IPs as Suspicious and block further connections from them
- The system must save blocked IPs to blocked_ips.txt so the block persists after restart
- The system must handle multiple simultaneous connections using threading
- The system must protect shared data using a threading Lock
- The system must shut down cleanly when the user presses Ctrl+C

6.2 User Requirements

- The user must be able to start the honeypot by running: `python honeypot.py`
- The user must be able to test the system using `client.py` and `client2.py`
- The user must be able to view real-time connection logs in the terminal
- The user must be able to open log.csv in Excel, LibreOffice, or IBM Cognos Analytics
- The user must be able to stop the server cleanly at any time using Ctrl+C

6.3 Environmental Requirements

- Python 3.8 or above must be installed
- No external Python libraries are required — all libraries are built into Python
- The system must run on any operating system that supports Python 3
- Port 8888 must be available and not blocked by a firewall

7. Design and Architecture

The system follows a simple but effective architecture with 5 core modules:

Module	Description
Server Initialization	Creates TCP socket, binds to 0.0.0.0:8888, starts listening
handle_client()	Core function: checks block list, counts attempts, logs data, sends response
IP Blocking & Persistence	Detects IPs with 5+ attempts, blocks them, saves to blocked_ips.txt
CSV Logging	Logs Time, IP, Port, Attempts, Status to log.csv after every connection
Threading & Lock	Handles multiple connections simultaneously with thread-safe data access

The process flow is as follows: the server starts and loads previously blocked IPs, then waits for connections. Each connection is handled in a new thread. The thread checks if the IP is blocked, increments the attempt count, flags suspicious IPs, logs the data, sends a fake response, and closes the connection.

8. Implementation

The complete implementation is in honeypot.py. Key sections are shown below:

Server Setup

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("0.0.0.0", 8888))
server.listen(5)
```

Threading

```
while True:
    client, addr = server.accept()

    thread = threading.Thread(target=handle_client, args=(client, addr))
    thread.start()
```

Threat Detection & Blocking

```
if attempts >= 5:  
    status = "Suspicious"  
    blocked_ips.add(ip)  
    with open(BLOCKED_IPS_FILE, "a") as f:  
        f.write(ip + "\n")
```

Fake Response

```
client.send("AgriSense IoT Gateway v2.1 - 403 Access Denied".encode())  
client.close()
```

9. Testing

9.1 Graphical User Interface (GUI) Layout

The system does not have a traditional graphical interface. The primary interface is the terminal/command line, which shows real-time connection logs in the following format:

```
=====
AgriSense Honeypot - Agriculture Threat Monitor
=====

Project : Honeypot Network for Agriculture Threat Intel
Port : 8888
Protocol : TCP (Socket)
Logging : log.csv
Blocking : blocked_ips.txt
Status : ACTIVE

=====

Waiting for connections...

2026-04-10 15:44:01 | 127.0.0.1:54321 | Attempts: 1 | Status: Normal
2026-04-10 15:44:02 | 127.0.0.1:54322 | Attempts: 2 | Status: Normal
2026-04-10 15:44:03 | 127.0.0.1:54323 | Attempts: 5 | Status: Suspicious
ALERT: Suspicious IP detected and blocked -> 127.0.0.1
Blocked connection attempt from 127.0.0.1
```

The IBM Cognos Analytics dashboard serves as the graphical interface for data analysis, displaying charts for Attempts by Time, Attempts by Status, and Attempts by IP Address.

9.2 Customer Testing

The system was tested using two test client scripts:

Test	Result
client.py — Single connection test	Server accepted connection, logged IP and time, sent fake response. Status: PASSED
client2.py — 7 repeated connections	First 4 attempts logged as Normal, 5th attempt triggered Suspicious alert and block. Status: PASSED
Blocked IP reconnection test	After being blocked, further connection attempts from same IP were immediately dropped. Status: PASSED
Server restart test	Blocked IPs loaded from blocked_ips.txt on restart — IP remained blocked. Status: PASSED
Graceful shutdown test	Ctrl+C produced clean shutdown message without crash. Status: PASSED

10. Evaluation

The project successfully meets all the requirements set out in the project brief. The honeypot correctly:

- Listens on port 8888 and accepts all incoming TCP connections
- Logs IP address, port, timestamp, and attempt count after every connection
- Sends a convincing fake AgriSense IoT Gateway denial message to attackers
- Detects suspicious IPs after 5 or more connection attempts
- Blocks suspicious IPs permanently and persists the block list across restarts
- Handles multiple simultaneous connections without data corruption
- Visualizes threat data effectively using IBM Cognos Analytics

The system goes beyond the basic project requirements by adding threading, thread-safe locking, persistent IP blocking, agriculture-specific branding, and a professional startup banner. These additions make the system more realistic and robust.

One limitation of the current system is that the `ip_count` dictionary resets on every restart — only the blocked IPs are persisted. A future improvement could also save the full attempt history to a database.

11. Snapshots of the Project

The following snapshots demonstrate the system in operation:

Snapshot 1 — Terminal Output

The terminal shows real-time connection logs including IP address, port, attempt count, and status. When the 5th attempt is detected, an alert is shown and the IP is blocked.

Snapshot 2 — log.csv

The CSV file stores all connection details. Each row contains the timestamp, IP address, port number, total attempts, and status (Normal or Suspicious). This file is used for analysis and visualization.

Snapshot 3 — IBM Cognos Dashboard

The IBM Cognos Analytics dashboard displays three visualizations from the log.csv data: Attempts by Time (line chart showing connection frequency over time), Attempts by Status (pie chart showing Normal vs Suspicious ratio), and Attempts by IP Address (bar chart showing total attempts per IP).

Snapshot 4 — blocked_ips.txt

The blocked IPs file stores the IP addresses of all suspicious clients. This file is read on every server startup to restore the block list, ensuring that previously blocked attackers cannot reconnect even after the server is restarted.

12. Conclusions

This project successfully demonstrates the concept of a Honeypot Network applied to an Agriculture IoT environment. Using only Python's built-in libraries, the system was able to:

- Simulate a vulnerable AgriSense IoT Gateway device on port 8888
- Monitor and log all incoming connection attempts in real-time
- Detect and block suspicious IPs using threshold-based threat detection
- Handle multiple simultaneous connections safely using threading and locks
- Persist blocked IPs across server restarts using a text file
- Visualize threat intelligence data using IBM Cognos Analytics

This project demonstrates core cybersecurity concepts including network monitoring, threat detection, logging, and data visualization in a practical and relevant agriculture context. It provides a solid foundation for understanding how real-world honeypot systems work.

13. Further Development or Research

The following improvements and extensions could be made in future versions of this project:

- Persist ip_count data across restarts by saving attempt counts to a database or JSON file
- Add email or SMS alerts when a suspicious IP is detected
- Simulate multiple agriculture IoT device types on different ports (e.g., 9999 for soil sensor, 7777 for irrigation controller)
- Add geolocation lookup to identify the geographic origin of attacking IPs
- Build a real-time web-based dashboard using Flask or Django instead of IBM Cognos
- Implement machine learning-based anomaly detection to identify more sophisticated attacks
- Deploy the honeypot on a cloud server (e.g., AWS, Google Cloud) to capture real-world attack data
- Add protocol simulation — respond with fake agriculture sensor data to further deceive attackers

14. References

- Python Official Documentation — <https://docs.python.org/3/library/socket.html>
- Python Threading Documentation — <https://docs.python.org/3/library/threading.html>
- Python CSV Documentation — <https://docs.python.org/3/library/csv.html>
- What is a Honeypot — <https://www.kaspersky.com/resource-center/threats/what-is-a-honeypot>
- IBM Cognos Analytics — <https://www.ibm.com/products/cognos-analytics>
- Forouzan, B.A., Data Communications and Networking, McGraw-Hill
- GitHub Repository — <https://github.com/farhanMether/Honeypot-Network-for-Agriculture-Threat-Intelligencz>

15. Appendix

A. Final honeypot.py Source Code

```
import socket
from datetime import datetime
import csv
import threading
import os

BLOCKED_IPS_FILE = "blocked_ips.txt"
lock = threading.Lock()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("0.0.0.0", 8888))
server.listen(5)
```

```
print("=" * 55)
print(" AgriSense Honeypot - Agriculture Threat Monitor")
print("=" * 55)
print(" Project : Honeypot Network for Agriculture Threat Intel")
print(" Port : 8888")
print(" Protocol : TCP (Socket)")
print(" Logging : log.csv")
print(" Blocking : blocked_ips.txt")
print(" Status : ACTIVE")
print("=" * 55)
print(" Waiting for connections...\n")

ip_count = {}
blocked_ips = set()

if os.path.exists(BLOCKED_IPS_FILE):
    with open(BLOCKED_IPS_FILE, "r") as f:
        blocked_ips = set(line.strip() for line in f if line.strip())
    print(f'[!] Loaded {len(blocked_ips)} previously blocked IPs\n')

if not os.path.exists("log.csv"):
    with open("log.csv", "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["Time", "IP Address", "Port", "Attempts", "Status"])
```

```
def handle_client(client, addr):  
    ip = addr[0]  
    port = addr[1]  
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")  
    with lock:  
  
        if ip in blocked_ips:  
            print(f'Blocked connection attempt from {ip}')            client.close()  
            return  
  
        if ip in ip_count:  
            ip_count[ip] += 1  
  
        else:  
            ip_count[ip] = 1  
            attempts = ip_count[ip]  
            status = "Normal"  
  
            if attempts >= 5:  
                status = "Suspicious"  
                blocked_ips.add(ip)  
                print(f'ALERT: Suspicious IP detected and blocked -> {ip}')                with open(BLOCKED_IPS_FILE, "a") as f:  
                    f.write(ip + "\n")
```

```
print(f'{timestamp} | {ip}:{port} | Attempts: {attempts} | Status: {status}')
```

```
with open("log.csv", "a", newline="") as file:
```

```
    writer = csv.writer(file)
```

```
    writer.writerow([timestamp, ip, port, attempts, status])
```

```
    client.send("AgriSense IoT Gateway v2.1 - 403 Access Denied".encode())
```

```
    client.close()
```



```
try:
```

```
    while True:
```

```
        client, addr = server.accept()
```

```
        thread = threading.Thread(target=handle_client, args=(client, addr))
```

```
        thread.start()
```

```
    except KeyboardInterrupt:
```

```
        print("\n[!] Honeypot shutting down gracefully...")
```

```
        server.close()
```

```
        print("[v] Server closed. Goodbye.")
```